

To Index or Not to Index: Optimizing Exact Maximum Inner Product Search

Firas Abuzaid, Geet Sethi, Peter Bailis, Matei Zaharia
Stanford DAWN Project

Abstract—Exact Maximum Inner Product Search (MIPS) is an important task that is widely pertinent to recommender systems and high-dimensional similarity search. The brute-force approach to solving exact MIPS is computationally expensive, thus spurring recent development of novel indexes and pruning techniques for this task. In this paper, we show that a hardware-efficient brute-force approach, blocked matrix multiply (BMM), can outperform the state-of-the-art MIPS solvers by over an order of magnitude, for some—but not all—inputs.

In this paper we also present a novel MIPS solution, MAXIMUS, that takes advantage of hardware efficiency *and* pruning of the search space. Like BMM, MAXIMUS is faster than other solvers by up to an order of magnitude, but again only for some inputs. Since no single solution offers the best runtime performance for *all* inputs, we introduce a new data-dependent optimizer, OPTIMUS, that selects online with minimal overhead the best MIPS solver for a given input. Together, OPTIMUS and MAXIMUS outperform state-of-the-art MIPS solvers by $3.2\times$ on average, and up to $10.9\times$, on widely studied MIPS datasets.

I. INTRODUCTION

Over the last decade, the Maximum Inner Product Search (MIPS) problem has increasingly become more important in machine learning and large-scale data systems. The classical MIPS setup is as follows: given a vector $\mathbf{u} \in \mathbb{R}^f$ and a second set of vectors I , where each $\mathbf{i} \in I$ is also in $\mathbb{R}^f$, find the vector $\mathbf{i}$ that maximizes the inner product $\mathbf{u}^T \mathbf{i}$. The problem generalizes even further, to finding the top K vectors in I that maximize $\mathbf{u}$.

MIPS has attracted interest because it is relevant to many real-world applications. For example, recommender systems—which are deployed in e-commerce [22], social networking [16], and media applications [19]—are often based on latent factor modeling using matrix factorization (MF) [36], [37]. In an MF model, each user is associated with a vector $\mathbf{u}$, and each item (e.g., a movie or song) is associated with a vector $\mathbf{i}$. The predicted rating for the item by the user is modeled as $\mathbf{u}^T \mathbf{i}$; therefore, solving MIPS in the context of an MF model effectively computes the top- K recommendations for a given user. Similarly, MIPS can be applied to high-dimensional similarity search [26] and multi-class prediction tasks [9], too.

For exact MIPS in the batch setting (i.e., compute top- K for all users at once), the naïve approach fails to scale to today’s data volumes, and two indexing techniques recently developed by the database community—LEMP [34] and FEXIPRO [21]—have proven to be the most efficient MIPS solvers and currently lead in terms of performance. The efficiency of these indexes is predicated on their ability to *i)* prune the

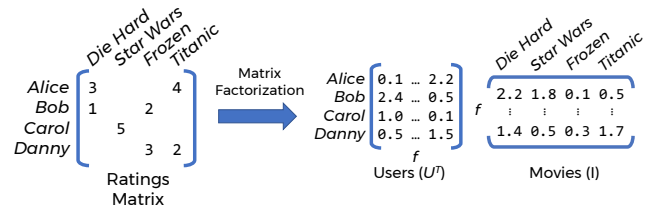


Fig. 1. Example of a matrix factorization (MF) model, a common setting where MIPS is applied to find the top- K items in I (i.e., movies) for each user $\mathbf{u} \in U$. In this paper, we focus on accelerating the end-to-end MIPS runtime for all users in U .

search space of candidate items, thus limiting the number of inner products computed, and *ii)* improve the efficiency of computing the inner products themselves.

However, in this paper, we show that both of these indexes do not always outperform a *hardware-efficient* brute-force approach: a dense matrix multiply of the user and item matrices using optimized, cache-efficient Basic Linear Algebra Subprogram (BLAS) libraries [6], [15], [38]. This approach, which we refer to as blocked matrix multiply (BMM) throughout this paper, has been applied to other areas of machine learning (such as model search [32]) and is surprisingly competitive with the state of the art. Prior indexing techniques for MIPS have not taken into account hardware optimizations that could be adopted by brute-force methods; in fact, they sometimes actually lose to the brute-force ones.

To illustrate this point, we trained 20 MF models on three gold-standard benchmark datasets and found that brute-force computation can outperform indexing on many of them, including several of the most accurate models. Figure 2 shows two examples. For the recommendation model trained on the Netflix Prize dataset (left), BMM outperforms the LEMP and FEXIPRO indexes by a factor of $1.9\text{--}3.1\times$. However, for the Yahoo R2 MF model (right), LEMP and FEXIPRO are $2\text{--}3.5\times$ faster than BMM.

Motivated by these experiments and others in Section V, we propose a new, hardware-optimized index called MAXIMUS, which captures the hardware efficiency of BMM while also pruning the search space, à la LEMP and FEXIPRO. MAXIMUS utilizes a combination of clustering, blocking, and linear algebra primitives from hardware-efficient software packages [15], [29] to achieve high performance while still being easy to implement. MAXIMUS first clusters users based on their weight vectors using k -means, then uses the cluster centroids to produce a sorted list of preferred items for each cluster. The cluster’s sorted list approximates the user’s pre-

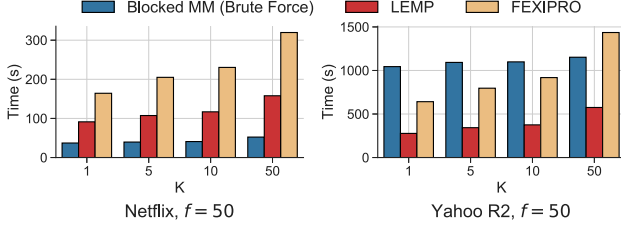


Fig. 2. Experiment comparing LEMP [34] and FEXIPRO [21], two state-of-the-art MIPS indexes, against a hardware-efficient brute-force approach, blocked matrix multiply. We measure the end-to-end top- K runtime for all users on two MF models from the Netflix Prize [4] and the Yahoo R2 [39] datasets. For Netflix, blocked MM is surprisingly fastest; for Yahoo R2, LEMP and FEXIPRO outperform it. In this paper, we develop OPTIMUS, an optimizer that chooses online the best MIPS serving strategy for a target model.

ferred items for every user assigned to the cluster. Then, rather than find each user’s top K items one at a time, MAXIMUS blocks the traversal of the sorted item lists for multiple users at a time, and applies a bound derived by Koenigstein et al. [18] to correct for the approximation. By combining hardware-efficient blocking with this pruning technique, MAXIMUS outperforms LEMP and FEXIPRO by $1.78\times$ on average, and up to $10.6\times$.

Despite the improvements that MAXIMUS gives us, it still does not beat BMM on all inputs. For exact MIPS, there is no single clear winner; a data-dependent strategy is needed. Therefore, we also propose OPTIMUS, a MIPS serving optimizer that automatically selects an efficient solving strategy for a particular input. To enable OPTIMUS to efficiently select the best strategy, we exploit two key observations. First, for the current MIPS indexes, index traversal is, in fact, much more expensive than index construction. For example, we show that the runtime to compute even the top $K = 1$ recommendation for all users can be $90\times$ greater than the runtime to build an index. Thus, we can cheaply construct an index to test its efficacy. Second, the performance of BMM and these MIPS indexes can be determined by measuring the runtime on a small number of users, then accurately extrapolating the performance on the entire model. This inspires a sampling-based approach, in which we construct a MIPS index and then directly compare its performance to BMM’s on a small sample of users. By applying an incremental t-test during this comparison, we can, for certain inputs, determine the best serving strategy without exploring the full sample, thereby reducing the runtime overhead. Overall, this online tuning step incurs low overhead (5.5% on average) while delivering high accuracy and large speedups across four different index types (up to $6\times$) that perform within 12% of an oracle-based optimizer with no overhead. These speedups are especially surprising given the highly-optimized nature of each index—for example, in their recent paper, LEMP improved performance over the previous best alternative exact index by up to $24\times$ [34].

We show that combining OPTIMUS and MAXIMUS yields a $3.2\times$ speedup on average (and up to $10.9\times$) compared to the state-of-the-art indexes alone. OPTIMUS and MAXIMUS

thus provide a pragmatic and easy-to-implement but difficult-to-beat baseline for MIPS across a wide range of inputs and query settings.

II. BACKGROUND

In this section, we provide background regarding the top- K MIPS problem, the surprising competitiveness of blocked matrix multiply (BMM), and the current state-of-the-art approaches to solving MIPS.

A. Problem Statement: Batch Exact MIPS

Let U be a set of user vectors $\mathbf{u} \in \mathbb{R}^f$, and let I be a set of item vectors $\mathbf{i} \in \mathbb{R}^f$, where f is referred to as the number of *latent factors*. For a single vector $\mathbf{u}$, the Maximum Inner Product Search problem is defined as

$$r_{ui} = \arg \max_{\mathbf{i} \in I} \mathbf{u}^T \mathbf{i}$$

In this paper, we focus on the *batch* top- K MIPS problem [27], [31], [34], where we wish to minimize the end-to-end runtime of finding the top K items that maximize the inner product for all $\mathbf{u} \in U$. However, MAXIMUS, our proposed index, can also accelerate MIPS for a subset of users at a time, as might happen in a model serving system like Clipper [7] that collects tens of requests at once.

Why Exact MIPS? While a range of existing techniques (Section VI) provide solutions to the *approximate* top- K retrieval problem, in this work, we are interested in evaluating MIPS quickly, subject to serving the *most accurate* results available. For some domains, especially revenue-critical applications that demand high-accuracy recommendations (e.g., e-commerce and advertising [30]), approximate techniques cannot be applied and exact MIPS is instead required. (Notably, the margin of accuracy separating the first- and second-place teams in the 2009 Netflix Prize challenge was only 0.01% [3].)

B. Blocked Matrix Multiply

The brute-force approach to MIPS is straightforward: for each user vector and each item vector, compute the inner product between the two. Once the ratings for all user-item pairs have been computed, select the top K items for each user (e.g., using a min-heap).

Each inner product can be computed using `sdot`, a standard BLAS library function [15], [38]. However, instead of repeatedly calling `sdot` in a double for-loop over the user and item vectors, we can replace the entire computation with a single matrix-matrix multiply between the users matrix and the items matrix (e.g., using the `sgeemm` function in BLAS), thereby blocking the entire computation of user-item ratings. We refer to this approach as blocked matrix multiply (BMM) throughout this paper.

In theory, replacing inner products with a single matrix-matrix multiplication does not change the runtime complexity of computing all user-item ratings; both are brute-force. Thus, if an index filters out a significant fraction of items, it should

be faster. Therefore, why is blocked matrix multiply often faster than indexes in practice?

The reason for this result is that, due to their popularity in numerical workloads, matrix-matrix multiply kernels are highly optimized for modern hardware. Specifically, modern linear algebra kernels will perform advanced data layout and blocking to maximize cache utilization, and, when available (e.g., on modern server-class processors), will aggressively vectorize code using SIMD instructions. These optimizations are applied automatically and transparently to the end programmer in BLAS libraries. By not designing for hardware efficiency, state-of-the-art MIPS indexes do not fully take advantage of these benefits found in modern hardware.

Thus, when we perform blocked matrix multiply using one of many implementations of linear algebra kernels, we benefit from decades of algorithmic and hardware-specific optimizations that yield substantial empirical speedups over naïve inner products (40×) or even matrix-vector multiply (20×). These speedups act as an effective “constant factor” improvement in runtime—they do not change the asymptotic performance, but, especially when evaluating MIPS on reference inputs, these constants have a significant effect on hardware efficiency.

C. Existing MIPS Indexes

MIPS is a topic of active research within the database community. In this section, we give an overview of prior approaches used to solve MIPS, which also provides necessary context for the remainder of this paper.

User Clustering. One of the first novel solutions to the MIPS problem was suggested by Koenigstein et al. [18], who proposed to cluster users via spherical clustering to perform *approximate* top- K queries. Effectively, they used the cluster centroids to serve as an approximation of the users’ preferences. To measure the approximation error, they derived a bound on the distortion of ratings based on the angle between the user vector and its assigned centroid. Here, we re-derive the bound (Equation 13 in [18]) as Equation 2 below and illustrate how to use this bound to approximate top- K scoring. (Note: This bound assumes that the user and item vectors reside in a metric space, a safe assumption for exact MIPS.)

Suppose we have a user vector $\mathbf{u}$, an item vector $\mathbf{i}$, and a centroid vector $\mathbf{c}$ that we have obtained from clustering the user vectors (i.e., $\mathbf{u}$ is assigned to the cluster represented by $\mathbf{c}$). Let θ_{ui} be the angle between $\mathbf{u}$ and $\mathbf{i}$, θ_{ic} be the angle between $\mathbf{i}$ and $\mathbf{c}$, and θ_{uc} be the angle between $\mathbf{u}$ and $\mathbf{c}$. Finally, let r_{ui} be the rating for the user-item pair $\mathbf{u}, \mathbf{i}$.

By the triangle inequality on angular distances in metric spaces, we have that

$$|\theta_{ic} - \theta_{uc}| \leq \theta_{ui} \leq \theta_{ic} + \theta_{uc}.$$

Thus, we can compute an upper bound on r_{ui} :

$$r_{ui} = \mathbf{u}^T \mathbf{i} = \|\mathbf{u}\| \|\mathbf{i}\| \cos(\theta_{ui}) \leq \|\mathbf{u}\| \|\mathbf{i}\| \max_{\theta \in [\theta_{ic} - \theta_{uc}, \theta_{ic} + \theta_{uc}]} \cos \theta.$$

For top- K , each user’s ratings are invariant under linear scaling. Thus, we can omit $\|\mathbf{u}\|$ to preserve the *relative*

ordering of items for a single user and obtain a linear scaling of r_{ui} , denoted r_{ui}^* such that $r_{ui}^* \|\mathbf{u}\| = r_{ui}$:

$$r_{ui}^* \leq \|\mathbf{i}\| \max_{\theta \in [\theta_{ic} - \theta_{uc}, \theta_{ic} + \theta_{uc}]} \cos \theta. \quad (1)$$

Since $\cos^{-1}(x) \in [0, \pi]$, we have two cases to consider. If $\theta_{uc} < \theta_{ic}$, then $\theta_{uc} < \pi$ and so Equation 1 is maximized at $\theta_{ic} - \theta_{uc}$. If $\theta_{uc} \geq \theta_{ic}$, then Equation 1 is maximized at the origin. Rewriting Equation 1, we have:

$$r_{ui}^* \leq \begin{cases} \|\mathbf{i}\| \cos(\theta_{ic} - \theta_{uc}) & \text{if } \theta_{uc} < \theta_{ic} \\ \|\mathbf{i}\| & \text{otherwise.} \end{cases} \quad (2)$$

Equation 2 provides a means of monotonically ranking items according to their angular distance from the cluster center. Later on, in Section III, we extend this inequality to perform *exact* queries using our proposed index, MAXIMUS.

LEMP. In SIGMOD 2015 [34] and TODS 2016 [33], Teflioudi et al. introduced the LEMP index, which empirically outperformed all prior approaches. LEMP solves the MIPS problem using a divide-and-conquer approach: first, it sorts the item vectors by length and partitions them into buckets, such that each bucket contains vectors of roughly equal magnitude. For each bucket, LEMP computes a set of candidate items for the top K by solving a smaller cosine similarity search problem; depending on the distribution of vector lengths, LEMP can choose one of several possible algorithms to retrieve these candidates, such as length-based pruning, angular-based pruning, incremental pruning (which leverages partial inner products and the Cauchy-Schwarz inequality), or naïve search (i.e., inner products). Crucially, LEMP does not consider blocked matrix multiply when scoring multiple users. LEMP chooses the retrieval algorithm by testing each method on a sample of user vectors. Once each bucket has been examined, LEMP merges the results by computing the ratings for each of the candidate item vectors using inner products and then finally selects the top K items. In an extensive study appearing in TODS 2016, LEMP was shown to outperform exact indexing alternatives by up to 24×.

We include LEMP as well as FEXIPRO [21] in our study as examples of state of the art in exact MIPS. Like LEMP, FEXIPRO does not consider blocked matrix multiply when scoring multiple users. To maximize the generality of our results, we instantiate our proposed optimization framework for both indexes (and MAXIMUS, which we propose), provide a head-to-head comparison of these indexes using the authors’ implementations, and determine the benefits of adaptive optimization for each in Section V.

III. A SIMPLE, HARDWARE-FRIENDLY INDEX

Our observation that BMM is competitive with state-of-the-art MIPS indexes on some—but not all—inputs, provides us with a number of lessons:

- The optimal choice of MIPS serving strategy is highly variable and input-dependent.

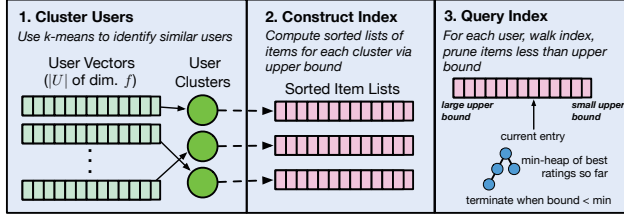


Fig. 3. Overview of clustering (Section III-A), index construction (Section III-B), and querying (III-C) in MAXIMUS.

- Hardware efficiency is critical in achieving maximum MIPS performance.
- We can leverage commodity hardware-optimized kernels to improve serving efficiency with limited effort.

Based on these insights, we develop a new index that relies heavily on commodity kernels for hardware-efficient operation, while also pruning the search space of candidate items. Our goal in developing this index is two-fold. First, we seek to synthesize a new algorithm that could constructively utilize the above lessons, combining them with existing ideas from the indexing literature. Second, we seek to develop a simple-to-understand and simple-to-implement but efficient index that could serve as a baseline both for our own empirical comparisons and for others to compare against in the future. The resulting index, which we call MAXIMUS, relies heavily on commodity analytics kernels and is implementable in just a few lines of pseudocode, yet is competitive with (and often exceeds the performance of) author-provided implementations of LEMP and FEXIPRO.

The remainder of this section introduces MAXIMUS in detail. We first begin with an overview of the MAXIMUS construction and querying techniques. Subsequently, we discuss the correctness of this index as well as practical implementation details and performance considerations.

- 1) **Cluster Users:** MAXIMUS clusters users into representative centroids that will approximate the users' preferences.
- 2) **Construct and Query Index:** MAXIMUS computes a conservative estimate of the maximum distortion between each cluster's predicted rating and the predicted ratings for users in the cluster. MAXIMUS subsequently uses this conservative upper bound to create a sorted list of items for each cluster.
- 3) **Walk Index:** To compute each user's top- K items, MAXIMUS walks the item list of the user's corresponding cluster, terminating when the previous bound implies there are no higher-ranked items to explore.

Figure 3 illustrates these steps, which we proceed to describe in detail. We first describe MAXIMUS's clustering strategy (Section III-A). We then show how MAXIMUS uses the cluster centroids to construct a prediction index (Section III-B) and can subsequently prune item vectors during exact top- K computation (Section III-C). We conclude with a discussion of optimizations for performance and runtime analysis (Sec-

Algorithm 1: MAXIMUS Index Construction and Querying

```

input: # clusters  $n$ , users  $U$ , items  $I$ 

initialization
   $L \leftarrow$  empty associative array of per-cluster sorted items

function CBound(vector  $\mathbf{c}$ , vector  $\mathbf{i}$ , angle  $\theta_b$ )
  return  $\|\mathbf{i}\| \cos(\theta_{ic} - \theta_b)$  if  $\theta_b < \theta_{ic}$  else  $\|\mathbf{i}\|$  ▷ Eqn. 3

function ConstructIndex()
  cluster  $U$  into clusters  $\mathcal{C} = \{C_1, \dots, C_n\}$  using  $k$ -means
  for cluster  $C_j \in \mathcal{C}$  having cluster centroid  $\mathbf{c}_j$  do
     $\theta_{bj} \leftarrow \max_{\mathbf{u} \in C_j} \cos^{-1} \left( \frac{\mathbf{u}^T \mathbf{c}_j}{\|\mathbf{u}\| \|\mathbf{c}_j\|} \right)$  ▷  $\max \theta_{uc}$  for  $C_j$ 
     $L[C_j] \leftarrow$  sort  $\mathbf{i} \in I$  by CBound( $\mathbf{c}_j, \mathbf{i}, \theta_{bj}$ ) descending

function QueryIndex(user  $\mathbf{u}$ , top- $K$  threshold  $K$ )
   $H$ : min-heap of maximum size  $K \leftarrow L[C_j][1 : K]$ 
  s.t.  $\mathbf{u} \in$  cluster  $C_j$  and each  $\mathbf{i} \in H$  is weighted by  $\mathbf{u}^T \mathbf{i}$ 
  for  $\mathbf{i} \in L[C_j][K+1 : ]$  do
    if CBound( $\mathbf{c}_j, \mathbf{i}, \theta_{bj}$ )  $< \min(H)$  then
      break
    else if  $\mathbf{u}^T \mathbf{i} > \min(H)$  then
      add  $\mathbf{i}$  to  $H$  with weight  $\mathbf{u}^T \mathbf{i}$ 
  return all items in  $H$ 

```

tion III-D). Algorithm 1 provides the entire pseudocode for the core MAXIMUS routines.

A. Clustering Users

MAXIMUS first partitions the set of users $U \in \mathbb{R}^f$ into a set of C clusters, with $C \ll |U|$. We will use a user $\mathbf{u}$'s assigned cluster centroid $\mathbf{c}$ as a means of finding an initial approximation of $\mathbf{u}$'s top- K items, which we then can iteratively refine per user to find the exact top K .

Choosing Clusters. As we previously illustrated in Equation 2, if we wish to provide an upper bound on the true rating r_{ui} using a centroid $\mathbf{c}$ instead of $\mathbf{u}$, then the quality of our bound will be determined by the angle between $\mathbf{c}$ and $\mathbf{u}$: the smaller θ_{uc} is, the tighter the approximation will be. The tighter the approximation, the more items we can potentially prune when we query the index for a user's top K . Therefore, we need to choose a clustering algorithm that ultimately minimizes θ_{uc} .

As prior work has shown [18], the ideal clustering algorithm to minimize θ_{uc} is spherical clustering, which projects the centroids onto the unit sphere per iteration and minimizes cosine dissimilarity rather than Euclidean distance. However, we found in our experiments that standard k -means reasonably approximates the target goal of minimizing angular distance, while also being computationally faster than spherical clustering. Because k -means is a common primitive, there are a number of widely available, hardware-efficient implementations. In our experiments, we found that, on average, the set of θ_{uc} s generated by k -means were only 7% greater than the same set generated by spherical clustering, while k -means was 2-3 $\times$ faster, leading to end-to-end runtime improvements of 5-10%. Therefore, we use k -means for user clustering in MAXIMUS, and report only results using k -means in Section V.

B. Constructing an Index

As described in Section II-C, Koenigstein et al. introduced the idea of clustering user vectors to evaluate approximate top- K queries, and showed how to bound the approximation error

based on the angle between the centroid and user vector (Equation 2). In this section, we extend this inequality to compute *exact* top- K queries, which forms the basis of MAXIMUS. In particular, we show how to use this approximation to prune items that cannot belong in $\mathbf{u}$'s top- K .

Instead of computing the upper bound for each user-item pair, we instead compute it for the *largest* such θ_{uc} contained in a given cluster, which we denote θ_b . Subsequently, we have:

$$r_{ci}^* \leq \begin{cases} \|\mathbf{i}\| \cos(\theta_{ic} - \theta_b) & \text{if } \theta_b < \theta_{ic} \\ \|\mathbf{i}\| & \text{otherwise.} \end{cases} \quad (3)$$

With this coarser approximation, MAXIMUS first computes r_{ci}^* for each centroid $\mathbf{c}$ and item $\mathbf{i}$ and, for each centroid $\mathbf{c}$, produces a list of items sorted by r_{ci}^* , denoted L_c . Both user clustering and this construction procedure are represented by the ConstructIndex procedure in Algorithm 1.

C. Performing Queries Using Centroid Index

Given the sorted centroid lists, we can now perform top- K queries for each user. First, we populate the min-heap H with the first K items from the user's centroid list L_c ; the items in H are weighted by their respective true ratings, r_{ui} . Then, we examine the remaining items in L_c by iteratively applying the upper bound in Equation 3. Each item's upper bound is then compared to the smallest r_{ui} in H . If the upper bound is less than the smallest r_{ui} , then that item and all subsequent items in L_c cannot have an r_{ui} greater than those already in H , since L_c is sorted in descending order by the upper bound, which is always greater than or equal to r_{ui} . Therefore, we can skip those remaining items and return the items in H as our top K . This procedure is represented by the QueryIndex procedure in Algorithm 1.

D. Hardware-Efficient Execution

As described at the start of this section, we designed MAXIMUS to leverage hardware-efficient libraries for linear algebra and advanced analytics. MAXIMUS uses k -means for clustering, of which there are many efficient implementations. After computing the upper bound in Equation 3, MAXIMUS uses efficient sorting routines, then walks the index during queries. This raises a natural question: is it possible to hardware-accelerate MAXIMUS's last step of index traversal?

Because each MAXIMUS cluster is shared across multiple users, we can block the first several steps of each walk. Specifically, for the first B items in a cluster list, we perform a blocked matrix multiply between all user vectors in the cluster and the first B item vectors in the cluster item list. This work sharing allows MAXIMUS's index traversal routine to make use of more matrix-matrix multiply operations (instead of less efficient matrix-vector or inner product operations) while still benefiting from MAXIMUS's early termination routines. If a user only needs to visit fewer than B items, this will result in wasted work. However, on balance, for modest blocking sizes, we find that sharing the first B items is beneficial to end-to-end runtime. We evaluate the impact of this optimization via a lesion study in Section V.

Combining Pruning and Hardware Efficiency. In Section V, we benchmark MAXIMUS on large U , I , and f , and yet we still find blocked matrix multiply to be competitive. Nevertheless, a natural concern with BMM is that, as model sizes grow, constant-factor runtime improvements due to hardware effects will diminish compared to an index, rendering matrix multiply much slower. That is, while an index can potentially prune irrelevant items if they are added to a dataset, BMM will compute them all. By combining the ability to algorithmically prune items *and* reap the benefit of hardware-efficient computation, MAXIMUS mitigates this concern; its index construction routine will place potentially highly-ranked items at the start of each cluster list, so the work of many users who are likely to prefer them can be accelerated via matrix-matrix multiply, without needing to visit *all* items.

Index Memory Requirement and Serving Runtime. For a U and I with f latent factors, the MAXIMUS index requires $O(|C||I|f)$ storage, with one sorted list of length $|I|$ per cluster. Given a k -means running time of $O(f|C||U|i)$ for i iterations and $\bar{w}$, the average number of items visited per user in Algorithm 1, MAXIMUS runs in time:

$$O(f(|C||U|i + |C||I|\log |I| + |U|\bar{w}\log K)), \quad (4)$$

where $|C||I|\log |I|$ captures the index construction time (including sorting) and $|U|\bar{w}\log K$ captures the time to walk each list. MAXIMUS is faster than brute force when Equation 4 is less than $O(f(|U||I| + |U||I|\log K))$; therefore, minimizing $\bar{w}$ is instrumental to MAXIMUS performance.

MAXIMUS Parameters. MAXIMUS's index exposes three parameters: the item blocking factor (B), the number of clusters ($|C|$), and the number of iterations to run k -means (i). All three of these parameters can be tuned to maximize performance; however, we found that MAXIMUS's runtime is robust across various settings of B , C , and i . After conducting a parameter sweep, we found that $B = 4096$, $|C| = 8$, and $i = 3$ is effective for many inputs. (Surprisingly, only a few iterations of k -means are needed to produce an adequate set of clusters.) In our evaluation, we report results with these three settings for all of our experiments.

E. Practical Usage

Because MAXIMUS clusters users rather than items, it assumes a relatively static set of users in the desired application. For applications with a dynamic set of users (assuming an initial static set is present), we can adapt MAXIMUS as follows: after generating our user clusters on the initial set of users, we forgo the clustering step for new users and simply assign them to the extant centroid that yields the smallest L_2 distance (i.e., perform only the assignment step in k -means). In our experiments, we found this strategy to be empirically effective for the collaborative filtering datasets used in our evaluation benchmarks: running k -means on a smaller sample of users (10%) and then assigning the remaining users to the resulting centroids did not impact the end-to-end runtime by more than 1%. In real applications, the churn in new users may

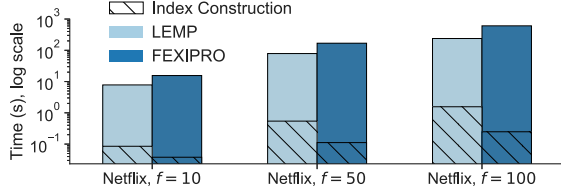


Fig. 4. Index construction time vs. end-to-end runtime for LEMP and FEXIPRO retrieving the top $K = 1$ items for all users. Index construction time is multiple orders of magnitude less than the retrieval time. (Note the log scale on the y-axis.)

reach a critical mass, and users can be removed as well as added. Therefore, periodically scheduling new rounds of user clustering to update the centroids is an interesting research question, which we leave as future work.

IV. OPTIMUS: A MIPS OPTIMIZER

Although MAXIMUS is, on average, an improvement on the state of the art for MIPS indexes, it still does not always outperform BMM for all inputs. We cannot rely solely on existing MIPS indexes for the fastest top- K serving strategy. Further, we cannot use rule-based principles strictly based on the weight vector characteristics of U and I to decide which serving technique is best; computing a measure of U and I 's "index-ability" is unlikely to be faster than computing the top K itself. Instead, multiple methods—including BMM—are necessary to serve a wide variety of inputs as efficiently as possible, and deciding between these techniques based on empirical cost measurements is critical. In response, we propose OPTIMUS, a new optimizer that automates the process of selecting a fast serving strategy.

Given weight vectors U and I as input, OPTIMUS's goal is to select the fastest serving strategy, choosing between either an exact indexing strategy or BMM. OPTIMUS is designed to operate in the online setting, where we have no a priori knowledge of the input model or underlying hardware except cache sizing. Therefore, OPTIMUS must run quickly—otherwise, the cost of making an optimization decision might dominate the cost of computing the top K using a single technique.

A. Online, Sample-Based Optimization

When operating online, the key idea in OPTIMUS is to estimate the overall performance of each serving technique based on a small sample of the users in U . Two factors make this strategy possible, and allow OPTIMUS to achieve high accuracy at low (often less than 5%) runtime overhead.

First, while top- K queries can be slow, requiring up to 14 hours to evaluate for all of the users in U in our measurements, the actual index construction time is relatively short for current indexing schemes, as is illustrated in Figure 4. In Section V, we show that the preprocessing and index construction overhead for FEXIPRO is on average 1.9%, for LEMP is on average 0.5%, and for MAXIMUS (Section III) is 1.5% in a batch scoring task that builds an index and then computes the top $K = 1$ prediction for every user. This ratio is even lower for larger K . Thus, when evaluating whether to use one of

these indexing methods, we can quickly construct a full index on U and/or I , and then evaluate its quality using a sample of the users (as we detail later). Note that, because index construction is inexpensive for the fastest indexing techniques in the literature, we currently always construct a full index.

Second, to evaluate the top- K performance of blocked matrix multiply, OPTIMUS extrapolates its performance on a small subset of users. Optimized matrix multiplication libraries like Intel MKL generally implement an $O(mnk)$ algorithm (for multiplying $m \times n$ and $n \times k$ matrices) that scales predictably once matrices are larger than the machine's L2 cache size, since they divide and process the matrices in blocks. Thus, OPTIMUS runs matrix-matrix multiplication for a subset of users and items that at least fully occupies the L2 cache and extrapolates performance from there. As we show in Section V, this approach accurately captures the performance of BMM. In addition, we also use the sample to extrapolate the runtime of extracting the actual top- K values (e.g., using a min-heap) once the ratings matrix has been computed.

Sample-based runtime estimation is a classic technique in data management systems, with previous applications spanning cardinality estimation [23], online query progress estimation [24], MapReduce [25], and cloud [11] databases. Perhaps closest to our proposal is [34], which uses an online optimizer to select between retrieval algorithms for each bucket of items. In OPTIMUS, we use a sampling-based optimizer to select between using an index *at all* and BMM—both logical operators for the same task of top- K search. Additionally, unlike [34], we empirically evaluate the runtime overhead, accuracy in obtaining these estimates, and effect on end-to-end MIPS serving performance in Section V.

In more detail, OPTIMUS performs online runtime estimation of indexing and BMM as follows: First, the optimizer constructs an index on U and/or I (or multiple indexes, if necessary). Second, OPTIMUS performs queries with a randomly chosen subset of user vectors and records the runtime. Third, the optimizer computes the rating predictions via blocked matrix multiply for a subset of user vectors and computes their top K . OPTIMUS subsequently estimates the total runtime for each method. Finally, if OPTIMUS was invoked in a batch prediction setting, it completes the top- K computation for the remaining users using the faster approach and reuses the results that it already had for the sampled users.

To implement the above steps, OPTIMUS must also decide on a sample size. For our target workloads, the number of users is large—at least 480,000. As a result, we can obtain high-quality estimates with only a small fraction of users—typically, 0.5%. However, especially for BMM and for hardware-optimized indexes such as LEMP, the sample size must be sufficiently large to demonstrate the benefits of hardware optimizations. For example, if we perform matrix-matrix multiply with only one user, we effectively perform matrix-vector multiply, which is substantially slower. Thus, the optimizer must ensure that the chosen sample is sufficiently large to illustrate hardware effects; in our optimizer implementation, we require that the sample size at least occupy

the entire L2 cache (256 KB in our experiments). For most U , this is easily occupied by a 0.5% sample—with 64-bit double-precision weights, 2048 u vectors with 100 latent factors requires 1.64MB of memory, easily occupying this requirement. Section V provides empirical measurements of the effect of sampling on runtime and accuracy.

Offline Performance Profiling for BMM. For blocked matrix multiply, we can alternatively develop an analytical cost model to predict offline (i.e., without sampling) the expected runtime performance of just *computing the user-item ratings*—but not the selection of the top K items. Because dense matrix multiply is compute-bound, we can model the runtime based on the total number of floating-point operations (FLOPs) needed and the FLOPs per cycle of the CPU [14].

In our experiments, we found that this analytical model was accurate within 5% of the measured dense matrix multiply runtimes in our evaluation. However, this model does not extend to the top- K selection stage of the MIPS problem: because the traversal through the min-heap depends on the distribution of ratings for each user, this data-dependent component of the computation is difficult to analytically model. While the dense matrix-matrix multiply operation does take the bulk of the computation runtime, the min-heap traversal time is non-negligible—at least 9.5% for our largest models. Therefore, we report results for OPTIMUS only using the online sampling approach in our evaluation.

Optimization: Early Stopping with t-test. Rather than measure the performance of both the index and blocked matrix multiple on the *entire* sample of users, we can, in some cases, stop early, by applying a one-sample t-test [12] on the per-user query times. That is, after measuring the performance of blocked matrix multiply, we can incrementally apply a one-sample t-test on the per-user times seen so far and compare it against the mean query time provided by BMM. If the calculated p-value of the test is less than a pre-determined threshold (e.g., 5%), then the optimizer can reject the null hypothesis and select either BMM or the index, whichever has the lower mean query time.

This technique, of course, will not work for indexes that also batch user queries for better performance; in those cases, the full sample has to be used to realize the full effects of the L2 cache. Therefore, OPTIMUS cannot employ this technique for indexes such as MAXIMUS. However, for indexes that do not batch users, the t-test is empirically effective for early stopping. For example, we found in our experiments that OPTIMUS needed to only examine 4% of the full sample of users when deciding between FEXIPRO and blocked matrix multiply for $K = 1$ on Netflix, $f = 10$.

B. Overhead of Optimizer

We can statically bound the overhead of OPTIMUS’s optimization routine. Denote the index construction time as C_I , the per-user index-based query time as Q_I , and the per-user BMM query time as M_I . Provided that we can estimate Q_I and M_I accurately for a sample fraction of users of size s (of

n total users), then the total runtime overhead of the optimizer is given by $C_I + \max(Q_I, M_I) \frac{s}{n}$.

Given that the cost of index construction is on average 1.5%, we can consider a few examples. If we use a 1% sample of users to evaluate our trade-off, and if $Q_I = M_I$, then the overhead due to optimization is approximately 1%. If Q_I and M_I differ by a factor of 3, the overhead of optimization compared to an oracle that automatically selects the right model is approximately 3%. However, compared to simply choosing the slower of the two methods, the total speedup is $\frac{300}{103} = 2.93\times$. More generally, given a d -times performance differential, the overhead will be $C_I + \frac{ds}{n}$. Given that the empirical performance differences we observe are regularly 2-3 $\times$, this overhead is relatively small compared to the benefit.

The above analysis is predicated on the ability to inexpensively and accurately estimate Q_I and M_I . In Section V, we demonstrate that sampling less than 1% of users results in high accuracy with an average of 6.3% overhead across four types of indexes.

V. EXPERIMENTAL EVALUATION

In this section, we empirically evaluate the index performance of MAXIMUS, the runtime efficiency of blocked matrix multiply, and the optimizer efficacy of OPTIMUS across various datasets and indexing methods found in the literature. Our results show that, across our reference datasets, OPTIMUS delivers, on average, a 2.8 $\times$ runtime improvement for LEMP, 1.8 $\times$ for MAXIMUS, a 5.2 $\times$ for FEXIPRO-SI, and 6 $\times$ improvement for FEXIPRO-SIR, within 8.8%, 11.2%, 11.1%, and 8.1% of an oracle optimizer, respectively. In addition, MAXIMUS is, on average, 1.78 $\times$ faster than LEMP and 4.1 $\times$ faster than FEXIPRO; when combined with OPTIMUS, MAXIMUS is 3.2 $\times$ faster than LEMP.

A. Experimental Setup

Datasets. We use four reference benchmark datasets for our experimental evaluation (Table I), including three collaborative filtering datasets. Of these, the Netflix dataset is by far the smallest (480K users and 17K items)—but arguably the best-studied in the literature for collaborative filtering. The Yahoo R2 dataset is the largest (1.8M users), and has not been benchmarked in prior work on MIPS serving; we select it to benchmark OPTIMUS’s scalability. The fourth dataset, GloVe-Twitter, contains high-dimensional (up to $f = 200$) word embeddings generated from a corpus of Tweets, and has been previously used to benchmark approximate nearest-neighbor and MIPS algorithms. Per [33], we use the same permutation to select user vectors from the dataset, and use the remaining vectors as item vectors.

Models. We evaluate top- K performance over a range of collaborative filtering models trained on these datasets, varying the number of latent factors and tuning the regularization for optimal accuracy.

TABLE I
DATASETS FOR EVALUATION

Dataset	# users	# items	# ratings
Netflix Prize [4] (Netflix)	480,189	17,770	100,480,507
Yahoo Music KDD [10] (KDD)	1,000,990	624,961	252,810,175
Yahoo Music R2 [39] (R2)	1,823,179	136,736	699,640,226
GloVe-Twitter [26]	100,000	1,093,514	–

To begin, the authors of LEMP [34] have made the models used in their evaluation publicly available;¹ for the Netflix dataset, these models were trained using Distributed Stochastic Gradient Descent, as described in [35], and we denote these models by $\star$ -DSGD throughout this section. For the Yahoo Music KDD dataset, the LEMP authors evaluated against the model found in [17], which is also considered one of the canonical reference models for this dataset. We denote this model by KDD-REF.

In addition to these models, we also train explicit feedback models (which incorporate the ratings made available in each dataset) using the NOMAD toolkit [40] (denoted $\star$ -NOMAD). We use the regularization parameter and hyperparameter settings reported in [40] as the starting point for a grid search for the optimal test RMSE. For the Netflix dataset, we also train an additional set of implicit feedback models [13], using Bayesian Personalized Ranking [28] as our training algorithm (denoted $\star$ -BPR). For the Yahoo R2 Music dataset, the literature did not contain any previously reported hyperparameter settings; therefore, we performed an expanded grid search of $\lambda = \{0, 1e^{-7}, 1e^{-6}, \dots, 1\}$. For all models (i.e., dataset and number of latent factors), we utilize the most accurate models with the lowest test RMSE.

Indexing Strategies. We compare three indexing strategies: LEMP, FEXIPRO, and MAXIMUS. Recent work from 2016 [34] shows that LEMP outperforms all prior solutions to the exact MIPS problem, providing a gold standard for indexes prior to its publication. In a more recent study, FEXIPRO outperforms LEMP, so we include these two indexing strategies as representative of the state of the art.

As described below, for LEMP and FEXIPRO, we utilize implementations provided by the authors of these studies. Our primary goal is not to directly compare the engineering quality of these indexes, but instead to understand *i*) their performance compared to hardware-optimized blocked matrix multiply, and *ii*) their amenability to online optimization via our proposed methods. Each index is implemented in C++ with double-precision floating-point arithmetic.

LEMP. We utilize the publicly available source code for LEMP provided by the authors.² We compile LEMP with SIMD optimizations enabled and tune LEMP’s parameters as instructed in the README. For the retrieval algorithm, we benchmark against LEMP-LI (for length-based and incremental pruning), which consistently achieves the best runtimes for top- K computation in [34].

FEXIPRO. We utilize the publicly available source code for FEXIPRO provided by the authors.³ As with LEMP, we compile FEXIPRO with SIMD optimizations enabled. In addition, we contacted the authors of FEXIPRO with the experimental results from this study to obtain guidance regarding parameter tuning. We tuned parameters per their guidance and report results from both FEXIPRO with all pruning strategies enabled (denoted FEXIPRO-SIR) and FEXIPRO with only SVD and integer pruning enabled (denoted FEXIPRO-SI).

MAXIMUS Implementation. We implement MAXIMUS in C++⁴ using double precision. We use the open-source Armadillo library [29] for k -means and Intel MKL [15] for MAXIMUS’s shared index traversal. We also compared with OpenBlas [38] and found limited effect on runtime for the relatively small shared index traversal operation.

BMM Implementation. We also compare to brute-force blocked matrix multiply using Intel MKL [15]. We compute ratings for users in a series of batches that each occupy the entirety of memory, and use a min-heap from the C++ standard library to compute the true top K .

Optimizer Implementation. We implement OPTIMUS from Section IV as a subroutine within each of the three indexes’ main routines. OPTIMUS performs sampling and runtime estimation, and then invokes either MKL or the regular indexing routine based on the output.

Environment. We report results from an Intel Xeon E7-4850 v3 2.20 GHz processor with 1 TB of RAM. We allow Intel MKL to use the entire RAM in blocking. Unless otherwise noted, we evaluate our benchmarks on a single core and report results in the single-threaded setting.

B. End-to-End Index Performance

In this section, we examine the end-to-end runtime—which includes index construction time—of MAXIMUS compared to LEMP, FEXIPRO, and BMM to compute the top K for all users. Figure 5 depicts the results across all four datasets and all 23 reference models; we report the runtimes for $K = \{1, 5, 10, 50\}$. Without using OPTIMUS, MAXIMUS is, on average, $1.8\times$ (and up to $10.6\times$) faster than LEMP and $> 10\times$ faster than both FEXIPRO-SI and FEXIPRO-SIR. Head to head, MAXIMUS is faster than LEMP 67% of the 92 model/top- K combinations. Compared to FEXIPRO-SI, MAXIMUS is faster for every single combination in our benchmarks except for one: KDD-NOMAD, $f = 50$, $K = 5$.

These results stand in contrast to [21], which reports significant speedups over LEMP. We have presented and discussed these results with the authors of both LEMP and FEXIPRO and, after conferring with them, believe these differences are due to several factors. First, unlike LEMP and MAXIMUS, FEXIPRO is optimized for the point query setting, in which a single user’s top K is queried at a time. Thus, FEXIPRO does not take advantage of hardware blocking, which could yield

¹<http://dws.informatik.uni-mannheim.de/en/resources/software/lemp>

²<https://github.com/uma-pi1/LEMP>

³<https://github.com/Hui-Li/MFRetrieval>

⁴<https://github.com/stanford-futuredata/optimus-maximus>

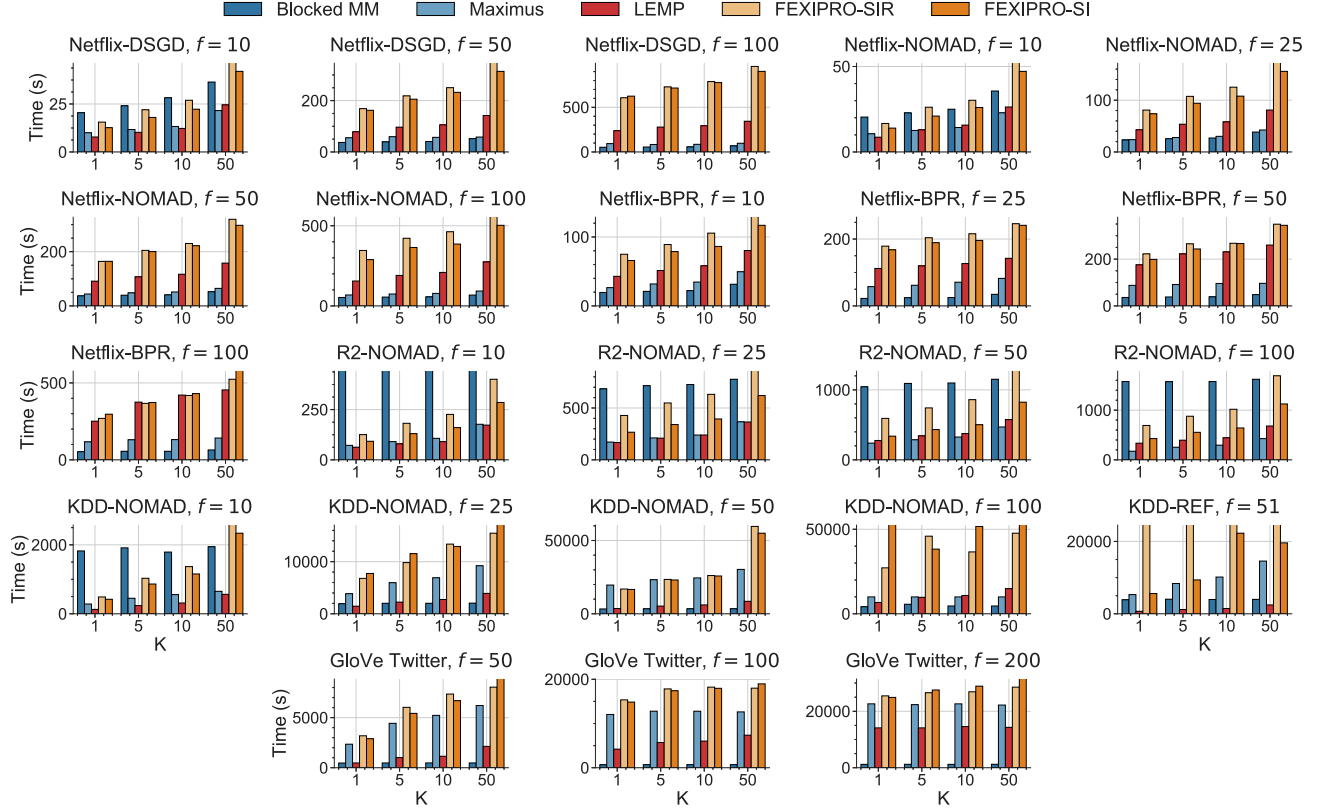


Fig. 5. MIPS wall-clock time of MAXIMUS and BMM compared to LEMP and FEXIPRO (lower is better). When combined with OPTIMUS, MAXIMUS is $3.2\times$ faster than LEMP, $7.3\times$ faster than FEXIPRO-SIR, and $6.3\times$ faster than FEXIPRO-SI.

additional performance benefits. In addition, [21] reports runtime relative to a custom implementation of LEMP, as opposed to the LEMP authors' implementation that we benchmark here. Finally, because the results presented here utilize models that have been regularized to obtain the lowest test RMSE, with the exception of the *-DSGD models, these models are not the same as those appearing in prior publications.

Compared to blocked matrix multiply, MAXIMUS is $2.7\times$ faster on average, but this speedup is not present for all models and values of K ; blocked matrix multiply is actually faster in 34.8% of the 92 model/top- K combinations in our benchmarks. (Note that the runtime for blocked matrix multiply varies with K , due to the time necessary to traverse the min-heap for all users.) The difference in runtime can also vary widely between the two: MAXIMUS can be up to $43.4\times$ faster than BMM for our reference models, but BMM can also be up to $18.7\times$ faster than MAXIMUS. This underscores the need for an optimizer that can choose between blocked matrix multiply and MAXIMUS (or other indexes) to achieve the best performance for MIPS.

Finally, note that, between MAXIMUS, BMM, and LEMP, no pair of techniques yielded the fastest runtime on all the reference models. LEMP is fastest on 11 of the 92 combinations, blocked matrix multiply is fastest on 53 of them, and MAXIMUS is fastest on the remaining 28. This indicates that a three-way optimizer that decides between LEMP, MAXIMUS,

and BMM is ideal. We investigate this question in Section V-C.

Multi-core Experiments. In addition to these single-threaded benchmarks, we also ran multi-core experiments to determine whether or not MAXIMUS and prior MIPS indexes would scale with increased parallelism. While blocked matrix multiply is trivial to parallelize, parallelizing the index structures of each respective system may not be as straightforward. The results are summarized in Figure 6: both MAXIMUS and LEMP achieved a near-linear speedup as we increase the number of cores from 1 to 16. Because both indexes are read-only, a simple partitioning scheme across users proves to be an effective parallelization strategy.

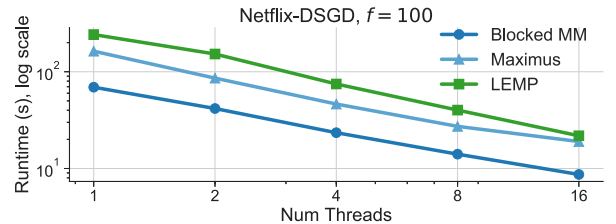


Fig. 6. End-to-end runtime of $K = 1$ query benchmarked on blocked matrix multiply, MAXIMUS, and LEMP across multiple cores. (FEXIPRO did not provide a multi-core implementation.) Both MAXIMUS and LEMP achieve near-linear speedups as we increase the number of cores.

TABLE II
EFFECTIVENESS OF ONLINE OPTIMIZER ON REFERENCE MODELS

Optimizer Choices	Accuracy	Avg. Overhead	Std. Dev. Overhead	Avg. speedup versus LEMP-only Baseline		
				Index Only	OPTIMUS (w/ overhead)	Oracle (no overhead)
BMM + LEMP	89.1%	4.3%	4.2%	1×	2.81×	3.08×
BMM + FEXIPRO-SI	97.8%	6.4%	8.1%	0.50×	2.60×	2.93×
BMM + FEXIPRO-SIR	97.8%	6.4%	7.8%	0.43×	2.56×	2.88×
BMM + MAXIMUS	93.5%	5.5%	5.9%	1.78×	3.15×	3.43×
BMM + LEMP + MAXIMUS	84.8%	9.1%	8.4%	-	2.99×	3.48×

C. OPTIMUS Efficacy and Overhead

To demonstrate the effectiveness and generality of OPTIMUS, we combine it with each of the three index techniques—MAXIMUS, LEMP, and FEXIPRO-SI/SIR—to choose between the given index and BMM on the 92 model/top- K combinations benchmarked in Figure 5. We also include an additional experiment in which we pair OPTIMUS with both MAXIMUS and LEMP, thus using OPTIMUS to perform a three-way optimization.

To effectively compare OPTIMUS’s runtime for a target model and K , we consider two baselines: first, we normalize our results by the runtime of the LEMP index only. Second, we consider the runtime resulting from consulting an oracle optimizer that always chooses the fastest strategy without incurring any runtime overhead. Table II summarizes the results of these experiments: for each two-way indexing strategy, OPTIMUS improves the top- K runtime and is within 8.3% of the optimal runtime efficiency compared to the oracle (average: 5.1%, including overhead). To observe the benefits of OPTIMUS, consider FEXIPRO-SIR, the slowest index that we measure in our benchmarks. Without OPTIMUS, FEXIPRO-SIR is, on average, 2.3× slower than LEMP on our reference models; however, with OPTIMUS, it becomes 2.6× faster than LEMP, within 11% of the maximum improvement that an oracle would provide. The other indexes exhibit similar trends, including MAXIMUS, which becomes 3.2× faster than LEMP—within 9% of the max—when paired with blocked matrix multiply using OPTIMUS.

The Accuracy measurement in Table II refers to OPTIMUS’s classification accuracy: how often does it choose the fastest serving strategy? For FEXIPRO and MAXIMUS, OPTIMUS effectively makes the correct choice with greater than 93% accuracy: in the case of MAXIMUS, only six model/top- K combinations are misclassified, while only two are misclassified for FEXIPRO-SI/SIR. These misclassifications do not lead to a significant runtime penalty: for the six misclassified examples for MAXIMUS, the runtimes of MAXIMUS and blocked matrix multiply are within a few seconds.

However, the accuracy for LEMP is less impressive: 10 of the 92 combinations are misclassified. To understand why this was the case, we ran an experiment in which we varied the user sample ratio used in OPTIMUS and measured the estimated runtime of the serving strategy (either blocked matrix multiply or an index) based on that sample. We measured the variance of the overall estimates for all of the indexes and BMM.

Figure 7 summarizes the results of our experiment on KDD-REF, $f = 51$, $K = 1$ for five distinct sample ratios ranging

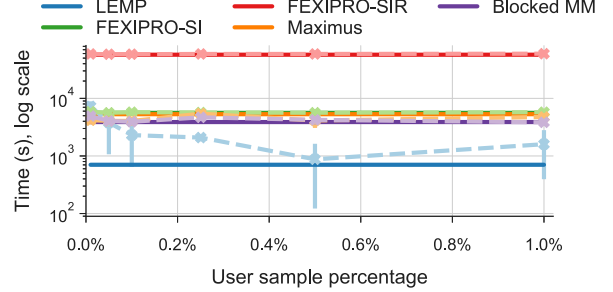


Fig. 7. OPTIMUS runtime estimates for all indexes on KDD-REF, $f = 51$, $K = 1$. Solid lines represent the true runtimes; dashed lines denote the estimates. Estimates are averaged over four runs; error bars represent the standard deviation. Despite the high variance of the estimates for LEMP, OPTIMUS makes the correct choice with a small ($< 1\%$) sample of users.

logarithmically from 0.01% to 1%. As depicted, OPTIMUS’s user sampling technique is robust and exhibits relatively low variance for MAXIMUS, BMM, and FEXIPRO, but the estimated runtimes for LEMP have much higher variance. This is because LEMP performs runtime adaptation of its serving strategy, and two separate samples of users may result in different pruning strategies (i.e., coordinate-wise versus L_2 -wise pruning; see Section II-C). For this particular model and choice of K , OPTIMUS is still able to make the right decision, since the estimated runtime for LEMP never exceeds the runtime estimate (or the true runtime) for BMM. However, for other models, this is not always the case, which explains why OPTIMUS’s accuracy with LEMP is lower. Nevertheless, combining OPTIMUS with LEMP is still beneficial, yielding a 2.8× speedup that is within 9% of the speedup obtained by the oracle.

The high variance exhibited by LEMP in our experiments also demonstrates why the three-way optimizer in the bottom row of the table—BMM + LEMP + MAXIMUS—actually achieves a smaller overall speedup (3×) than BMM + MAXIMUS (3.2×). In addition to the slight reduction in accuracy (84.8%), OPTIMUS begins to incur a higher runtime overhead—9.1% on average—since it now has to construct and query multiple indexes. For this three-way comparison OPTIMUS’s speedup is within 15% of the max possible speedup, 3.48×

We were initially surprised to find that absolute optimizer accuracy was not a robust signal of end-to-end speedup. The three-way optimizer demonstrates this well: with 84.8% accuracy, OPTIMUS is still within 15% of the optimal speedup on average. The primary reason for this phenomenon is that, in the cases where OPTIMUS chooses a sub-optimal strategy,

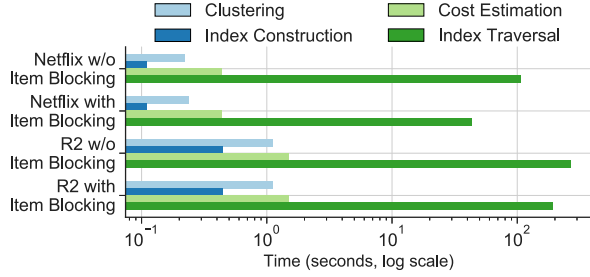


Fig. 8. Runtime breakdown of MAXIMUS for $K = 1$ on Netflix-NOMAD, $f = 50$ and R2-NOMAD, $f = 50$. Enabling item blocking improves throughput by $2.4\times$ and $1.4\times$, respectively.

the margin of difference is often small. For example, in R2-NOMAD $f = 50$, $K = 1$ (Figure 5), MAXIMUS and LEMP are remarkably close—within 12%—while each index is considerably faster than BMM—a factor of $3.75\times$. Thus, unless OPTIMUS’s runtime estimate is off by more than $3.75\times$, choosing the slower index has limited impact on this model. This result suggests that even coarse-grained runtime estimates are useful in accelerating these indexing workloads.

Runtime Analysis of MAXIMUS and Item Blocking Lesion Study. Finally, to better understand the impact of each stage of MAXIMUS’s execution, we measured the running time of each component. On average, MAXIMUS incurs an overhead of 1.8% for clustering, constructing its index, and performing cost estimation. We illustrate a breakdown for Netflix-NOMAD, $f = 50$ and R2-NOMAD, $f = 50$ in Figure 8, both of which use MAXIMUS’s index; for Netflix, MAXIMUS spends 0.79 seconds in the first three stages, and over 43 seconds in the final stage when computing predictions. We also illustrate the effect of hardware-efficient item blocking, which delivers $2.4\times$ and $1.4\times$ speedups for these datasets; the effect for Netflix is more pronounced because the average number of items visited in the index for each user (i.e., $\bar{w}$) is larger than in Yahoo R2. Sharing a single, small blocked matrix multiply at the start of index traversal allows MAXIMUS to benefit from hardware-efficient BLAS. Overall, MAXIMUS’s overheads are small, especially relative to the speedups that OPTIMUS enables by choosing between MAXIMUS and blocked matrix multiply.

VI. RELATED WORK

State of the Art: LEMP and FEXIPRO. LEMP [34] (previously described in Section II-C) and FEXIPRO [21] are the closest related work in exact MIPS, which we build upon and evaluate against in this work. Here, we first summarize the key contributions of FEXIPRO, then discuss key differences between these two indexes and our work.

FEXIPRO [21] leverages three pruning strategies for the top- K MIPS problem: *i*) applying singular value decomposition to the input to only compute partial inner products, *ii*) applying input quantization to perform integer-based operations only, and *iii*) applying a non-negative transformation on the input to ensure monotonicity, further promoting pruning. In their paper, the FEXIPRO authors report that these three strategies combined together achieve an order-of-magnitude

speedup over a custom point-query implementation of LEMP. As discussed in Section V-B, our results differ on the models we evaluated.

There are several differences between the prior work on MIPS serving and ours. First, both LEMP and FEXIPRO do not consider bypassing the index search altogether to use blocked matrix multiply, which we show is often better than their performance. Neither LEMP nor FEXIPRO considers BMM as a strategy when scoring a group of users, and their data structures are not designed to enable BMM during index traversal. While LEMP samples users to select the best retrieval algorithm per bucket, OPTIMUS’s sampling strategy answers a much coarser granularity question: should a given model be indexed at all? To minimize the overhead of sampling, we apply an incremental t-test for early stopping, another difference between our approach and LEMP’s. Lastly, both LEMP and FEXIPRO build an index on the items, rather than on the users.

User Clustering and Other Indexes. Several alternative methods propose tree-based indexes for MIPS. As discussed in Section II-C, Koenigstein et al. [18] introduce a method for computing *approximate* top- K recommendations via user clustering. In their method, the user vectors are clustered using spherical clustering and the top K items are pre-computed for each cluster—the top- K for a given user is the top- K of the cluster the user belongs to. In their analysis, they provide the bound we use in Equation 2 in Section III, but they use it for the purpose of *approximate*—not exact—top- K . In MAXIMUS, we show how to utilize this bound to build an efficient index for exact MIPS.

Ram and Gray [27] present three different techniques for indexing item vectors: single-ball trees, dual-ball trees, and cone trees. All three data structures share a similar strategy: they recursively subdivide the metric space of items into hyperspheres. Every node in the tree represents a set of points, and each node is indexed with a center and a ball enclosing all the points in the node. Of the three, the cone tree offers the fastest speedups. Follow-on work by Curtin et al. [8] extends this method using cover trees [5], but Teflioudi et al. [34] show that these methods are slower than LEMP.

Approximate MIPS. A large body of work considers the approximate setting for MIPS, whereby search procedures attain approximations of the true top K . Shrivastava et al. [31] use asymmetric hash functions in their LSH subroutine, which reduce the approximate MIPS problem to a sublinear nearest-neighbor search. Similarly, Bachrach et al. [1] also reduce MIPS to NNS using a novel Euclidean transformation, which allows users to trade off top- K accuracy for better performance. Our focus in this work is the exact setting—delivering the most accurate predictions possible with the fastest speed.

Model Serving. Model serving systems are of increasing practical importance in the database community [2], [20]. Prior systems, such as TUPAC [32], have shown that dense matrix multiply is an effective approach for model/parameter search, while other systems, such as Clipper [7], demonstrate that

adaptive batching can lead to performance wins in online serving. In our work, we show that, *for certain models*, blocked matrix multiply is actually faster than state-of-the-art MIPS indexes, which prune significant computation but do not take advantage of hardware efficiency. Thus, we introduce a new MIPS index, MAXIMUS, that takes advantage of high-performance BLAS libraries while also pruning computation. Because the best serving strategy varies from model to model, we introduce OPTIMUS, which determines the best serving strategy using an online, sampling-based approach.

VII. CONCLUSION

MIPS is a critical component of many modern workloads, including recommender systems and information extraction tasks. In this work, we show that the fastest of today's indexes do not always outperform blocked matrix multiply. We thus propose MAXIMUS, a simple but efficient indexing scheme that leverages linear algebra kernels to gain hardware efficiency while also pruning computation. In addition, we design OPTIMUS, a system that can efficiently choose between using an index and blocked matrix multiply. Together, OPTIMUS and MAXIMUS achieve speedups of $3.2\times$ on average, and up to $10.9\times$, on popular word embeddings and well-tuned models from recommendation datasets.

VIII. ACKNOWLEDGMENTS

This research was supported in part by affiliate members and other supporters of the Stanford DAWN project—Ant Financial, Facebook, Google, Infosys, Intel, Microsoft, NEC, SAP, Teradata, and VMware—as well as Toyota Research Institute, Keysight Technologies, Hitachi, Northrop Grumman, Amazon Web Services, Juniper Networks, NetApp, and the NSF under CAREER grant CNS-1651570.

REFERENCES

- [1] Y. Bachrach, Y. Finkelstein, R. Gilad-Bachrach, L. Katzir, N. Koenigstein, N. Nice, and U. Paquet. Speeding up the Xbox recommender system using a Euclidean transformation for inner-product spaces. In *RecSys*, pages 257–264. ACM, 2014.
- [2] D. Baylor, E. Breck, H.-T. Cheng, N. Fiedel, C. Y. Foo, Z. Haque, S. Haykal, M. Ispir, V. Jain, L. Koc, et al. TFX: A TensorFlow-Based Production-Scale Machine Learning Platform. In *KDD*, pages 1387–1395. ACM, 2017.
- [3] R. M. Bell and Y. Koren. Lessons from the Netflix prize challenge. *SIGKDD Explorations Newsletter*, 9(2):75–79, 2007.
- [4] J. Bennett, S. Lanning, et al. The Netflix prize. In *KDD Cup and Workshop*, volume 2007, page 35. New York, NY, USA, 2007.
- [5] A. Beygelzimer, S. Kakade, and J. Langford. Cover trees for nearest neighbor. In *ICML*, pages 97–104. ACM, 2006.
- [6] L. S. Blackford, A. Petitet, R. Pozo, K. Remington, R. C. Whaley, J. Demmel, J. Dongarra, I. Duff, S. Hammarling, G. Henry, et al. An updated set of basic linear algebra subprograms (BLAS). *TOMS*, 28(2):135–151, 2002.
- [7] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica. Clipper: A Low-Latency Online Prediction Serving System. In *NSDI*, pages 613–627, 2017.
- [8] R. R. Curtin, P. Ram, and A. G. Gray. Fast exact max-kernel search. In *ICDM*, pages 1–9. SIAM, 2013.
- [9] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, pages 248–255. IEEE, 2009.
- [10] G. Dror, N. Koenigstein, Y. Koren, and M. Weimer. The Yahoo! Music Dataset and KDD-Cup'11. 2012.
- [11] J. Duggan, Y. Chi, H. Hacigumus, S. Zhu, and U. Cetintemel. Packing light: Portable workload performance prediction for the cloud. In *ICDEW*, pages 258–265. IEEE, 2013.
- [12] D. A. Freedman. *Statistical models: theory and practice*. Cambridge University Press, 2009.
- [13] Y. Hu, Y. Koren, and C. Volinsky. Collaborative filtering for implicit feedback datasets. In *ICDM*, pages 263–272. IEEE, 2008.
- [14] R. Iakymchuk and P. Bientinesi. Execution-less performance modeling. In *PMBS*, pages 11–12. ACM, 2011.
- [15] Intel Math Kernel Library. <https://software.intel.com/en-us/intel-mkl>.
- [16] M. Kbailjo and A. Ilic. Recommending items to more than a billion people. <https://code.facebook.com/posts/861999383875667>, 2015.
- [17] N. Koenigstein, G. Dror, and Y. Koren. Yahoo! music recommendations: modeling music ratings with temporal dynamics and item taxonomy. In *RecSys*, pages 165–172. ACM, 2011.
- [18] N. Koenigstein, P. Ram, and Y. Shavitt. Efficient retrieval of recommendations in a matrix factorization framework. In *CIKM*, pages 535–544. ACM, 2012.
- [19] Y. Koren, R. Bell, C. Volinsky, et al. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.
- [20] A. Kumar, R. McCann, J. Naughton, and J. M. Patel. A Survey of the Existing Landscape of ML Systems. *UW-Madison CS Tech. Rep. TR1827*, 2015.
- [21] H. Li, T. N. Chan, M. L. Yiu, and N. Mamoulis. FEXIPRO: Fast and Exact Inner Product Retrieval in Recommender Systems. In *SIGMOD*, 2017.
- [22] G. Linden, B. Smith, and J. York. Amazon.com recommendations: Item-to-item collaborative filtering. *IEEE Internet Computing*, 7(1):76–80, 2003.
- [23] R. J. Lipton and J. F. Naughton. Query size estimation by adaptive sampling. In *PODS*, pages 40–46. ACM, 1990.
- [24] C. Mishra and N. Koudas. A lightweight online framework for query progress indicators. In *ICDE*, pages 1292–1296. IEEE, 2007.
- [25] K. Morton, A. Friesen, M. Balazinska, and D. Grossman. Estimating the progress of MapReduce pipelines. In *ICDE*, pages 681–684. IEEE, 2010.
- [26] J. Pennington, R. Socher, and C. Manning. Glove: Global vectors for word representation. In *EMNLP*, pages 1532–1543, 2014.
- [27] P. Ram and A. G. Gray. Maximum inner-product search using cone trees. In *KDD*, pages 931–939. ACM, 2012.
- [28] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. In *AUAI*, pages 452–461. AUAI Press, 2009.
- [29] C. Sanderson and R. Curtin. Armadillo: a template-based C++ library for linear algebra. *Journal of Open Source Software*, 1(2):26–32, 2016.
- [30] J. B. Schafer, J. Konstan, and J. Riedl. Recommender systems in e-commerce. In *EC*, pages 158–166. ACM, 1999.
- [31] A. Shrivastava and P. Li. Asymmetric LSH (ALSH) for sublinear time maximum inner product search (MIPS). In *NIPS*, pages 2321–2329, 2014.
- [32] E. R. Sparks, A. Talwalkar, D. Haas, M. J. Franklin, M. I. Jordan, and T. Kraska. Automating model search for large scale machine learning. In *SOCC*, pages 368–380. ACM, 2015.
- [33] C. Teflioudi and R. Gemulla. Exact and Approximate Maximum Inner Product Search with LEMP. volume 42, page 5. ACM, 2017.
- [34] C. Teflioudi, R. Gemulla, and O. Mykytiuk. Lemp: Fast retrieval of large entries in a matrix product. In *SIGMOD*, pages 107–122. ACM, 2015.
- [35] C. Teflioudi, F. Makari, and R. Gemulla. Distributed matrix completion. In *ICDM*, pages 655–664. IEEE, 2012.
- [36] A. Van den Oord, S. Dieleman, and B. Schrauwen. Deep content-based music recommendation. In *NIPS*, pages 2643–2651, 2013.
- [37] H. Wang, N. Wang, and D.-Y. Yeung. Collaborative deep learning for recommender systems. In *KDD*, pages 1235–1244. ACM, 2015.
- [38] Z. Xianyi, W. Qian, and W. Saar. OpenBLAS: an optimized BLAS library, 2016.
- [39] Yahoo! Research. R2 - Yahoo! Music User Ratings of Songs with Artist, Album, and Genre Meta Information, v. 1.0. <https://webscope.sandbox.yahoo.com/>.
- [40] H. Yun, H.-F. Yu, C.-J. Hsieh, S. Vishwanathan, and I. Dhillon. NOMAD: Non-locking, stochastic Multi-machine algorithm for Asynchronous and Decentralized matrix completion. *VLDB*, 7(11):975–986, 2014.